Matthew Anderson

Professor Maumalanga

CSP 31A 30386

19 March 2026

Lab 8B Payroll Analyzer

Output:

```
Loaded employees.txt

MENU:
  1. Display all employees (original order)
  2. Sort by name (A-Z)
  3. Sort by gross pay (highest first)
  4. Sort by department
  5. View department payroll breakdown
  6. View payroll summary
  7. Exit
Enter your choice (1-7): _
```

```
Enter your choice (1-7): 1


EMPLOYEE PAYROLL REPORT
======================================================================
   ID        Name    Dept   Reg Hrs   OT Hours      Rate   Gross Pay
----------------------------------------------------------------------
 1001     Johnson   SALES      40.0        5.0    $25.50    $1211.25
 1002    Williams     ENG      40.0        0.0    $45.00    $1800.00
 1003      Garcia     MKT      38.0        0.0    $28.75    $1092.50
 1004    Martinez     ENG      40.0       12.0    $52.00    $3016.00
 1005    Robinson      HR      40.0        0.0    $32.50    $1300.00
 1006       Clark   SALES      40.0        4.0    $27.00    $1242.00
 1007       Lewis     ENG      40.0        8.0    $48.50    $2522.00
 1008         Lee      IT      40.0        0.0    $55.00    $2200.00
 1009      Walker     MKT      36.0        0.0    $26.25    $945.00
 1010        Hall      HR      40.0        2.0    $35.00    $1505.00
 1011       Allen      IT      40.0        5.0    $50.00    $2375.00
 1012       Young   SALES      40.0        0.0    $24.00    $960.00
```

```
Enter your choice (1-7): 2


EMPLOYEES SORTED BY NAME (A-Z)
=============================================================================
   ID        Name    Dept   Reg Hrs   OT Hours      Rate   Gross Pay
-----------------------------------------------------------------------------
 1011       Allen      IT      40.0        5.0    $50.00    $2375.00
 1006       Clark   SALES      40.0        4.0    $27.00    $1242.00
 1003      Garcia     MKT      38.0        0.0    $28.75    $1092.50
 1010        Hall      HR      40.0        2.0    $35.00    $1505.00
 1001     Johnson   SALES      40.0        5.0    $25.50    $1211.25
 1007       Lewis     ENG      40.0        8.0    $48.50    $2522.00
 1008         Lee      IT      40.0        0.0    $55.00    $2200.00
 1004    Martinez     ENG      40.0       12.0    $52.00    $3016.00
 1005    Robinson      HR      40.0        0.0    $32.50    $1300.00
 1002    Williams     ENG      40.0        0.0    $45.00    $1800.00
 1009      Walker     MKT      36.0        0.0    $26.25    $945.00
 1012       Young   SALES      40.0        0.0    $24.00    $960.00
```

```
Enter your choice (1-7): 3


EMPLOYEES SORTED BY GROSS PAY (Highest to Lowest)
=============================================================================
   ID        Name    Dept   Reg Hrs   OT Hours      Rate   Gross Pay
-----------------------------------------------------------------------------
 1004    Martinez     ENG      40.0       12.0    $52.00    $3016.00
 1002    Williams     ENG      40.0        0.0    $45.00    $1800.00
 1006       Clark   SALES      40.0        4.0    $27.00    $1242.00
 1011       Allen      IT      40.0        5.0    $50.00    $2375.00
 1005    Robinson      HR      40.0        0.0    $32.50    $1300.00
 1007       Lewis     ENG      40.0        8.0    $48.50    $2522.00
 1008         Lee      IT      40.0        0.0    $55.00    $2200.00
 1010        Hall      HR      40.0        2.0    $35.00    $1505.00
 1001     Johnson   SALES      40.0        5.0    $25.50    $1211.25
 1003      Garcia     MKT      38.0        0.0    $28.75    $1092.50
 1012       Young   SALES      40.0        0.0    $24.00    $960.00
 1009      Walker     MKT      36.0        0.0    $26.25    $945.00
```

```
Enter your choice (1-7): 4


EMPLOYEES SORTED BY DEPARTMENT (A-Z)
=================================================================================
   ID        Name    Dept   Reg Hrs   OT Hours      Rate   Gross Pay
---------------------------------------------------------------------------------
 1002    Williams     ENG      40.0        0.0    $45.00    $1800.00
 1004    Martinez     ENG      40.0       12.0    $52.00    $3016.00
 1007       Lewis     ENG      40.0        8.0    $48.50    $2522.00
 1005    Robinson      HR      40.0        0.0    $32.50    $1300.00
 1010        Hall      HR      40.0        2.0    $35.00    $1505.00
 1008         Lee      IT      40.0        0.0    $55.00    $2200.00
 1011       Allen      IT      40.0        5.0    $50.00    $2375.00
 1009      Walker     MKT      36.0        0.0    $26.25    $945.00
 1003      Garcia     MKT      38.0        0.0    $28.75    $1092.50
 1001     Johnson   SALES      40.0        5.0    $25.50    $1211.25
 1006       Clark   SALES      40.0        4.0    $27.00    $1242.00
 1012       Young   SALES      40.0        0.0    $24.00    $960.00
```

```
Enter your choice (1-7): 5


PAYROLL BY DEPARTMENT
================================================================================
   ID        Name           Reg Hrs   OT Hours      Rate   Gross Pay
--------------------------------------------------------------------------------
Department: ENG
--------------------------------------------------------------------------------
 1002    Williams             40.0        0.0    $45.00    $1800.00
 1004    Martinez             40.0       12.0    $52.00    $3016.00
 1007       Lewis             40.0        8.0    $48.50    $2522.00
                                       Department total: $7338.00
Department: HR
--------------------------------------------------------------------------------
 1005    Robinson             40.0        0.0    $32.50    $1300.00
 1010        Hall             40.0        2.0    $35.00    $1505.00
                                       Department total: $2805.00
Department: IT
--------------------------------------------------------------------------------
 1008         Lee             40.0        0.0    $55.00    $2200.00
 1011       Allen             40.0        5.0    $50.00    $2375.00
                                       Department total: $4575.00
Department: MKT
--------------------------------------------------------------------------------
 1009      Walker             36.0        0.0    $26.25    $945.00
 1003      Garcia             38.0        0.0    $28.75    $1092.50
                                       Department total: $2037.50
Department: SALES
--------------------------------------------------------------------------------
 1001     Johnson             40.0        5.0    $25.50    $1211.25
 1006       Clark             40.0        4.0    $27.00    $1242.00
 1012       Young             40.0        0.0    $24.00    $960.00
                                       Department total: $3413.25
================================================================================
Company Total: $20168.75
```

```
Enter your choice (1-7): 6


PAYROLL SUMMARY REPORT
================================================================================

  EMPLOYEE STATISTICS
----------------------------------
  Total Employees: 12  Employees with Overtime: 6
  Employees without OT: 6

  HOURS SUMMARY
----------------------------------
  Total Regular Hours: 474.0
  Total Overtime Hours: 36.0
  Total Hours Worked: 510.0
  Average Hours/Employee: 42.5

  PAY SUMMARY
----------------------------------
  Total Regular Pay: $17817.50
  Total Overtime Pay: $2351.25
  Total Gross Payroll: $20168.75
  Highest Paid: Martinez $3016.00
  Lowest Paid: Walker $945.00
  Average Pay: $1680.73

  DEPARTMENT BREAKDOWN
----------------------------------
  ENG: 3 employees $7338.00 (36.38%)
  HR: 2 employees $2805.00 (13.91%)
  IT: 2 employees $4575.00 (22.68%)
  MKT: 2 employees $2037.50 (10.10%)
  SALES: 3 employees $3413.25 (16.92%)
```

Code (`Matthew_Anderson_PayrollAnalyzer.cpp`):

```
 // Matthew Anderson - Payroll Analyzer - 3/25/2025
// Much of the code is from my previous lab assignment but adapted to use
with vectors
```

```
#include <iostream>
#include <fstream>
#include <vector>
#include <array>
#include <algorithm>
#include <chrono>
#include <iomanip>

using namespace std;

const int SORT_NAME_ASCENDING = 1;
const int SORT_PAY_DECENDING = 2;
const int SORT_DEPARTMENT_ASCENDING = 3;
const int SORT_ID_ASCENDING = 4;

const string DATA_FILE = "employees.txt";
const auto DOUBLE_BAR = string(70, '=');
const auto BAR = string(70, '-');
const auto BAR_SMALL = string(35, '-');
const double OVERTIME_MULTIPLIER = 1.5;

const vector<string> MENU_ITEMS = {
    "Display all employees (original order)",
    "Sort by name (A-Z)",
    "Sort by gross pay (highest first)",
    "Sort by department",
    "View department payroll breakdown",
    "View payroll summary",
    "Exit",
};

int readInt(string prompt, int min, int max);
void displayMenu(const vector<string> &menu_items);
bool loadEmployeeData(vector<int> &ids, vector<string> &names, vector<string>
&depts,
                      vector<double> &hours, vector<double> &rates, string
filename);
void sortEmployees(vector<int> &ids, vector<string> &names, vector<string>
&depts,
          vector<double> &hours, vector<double> &rates, int sortStyle);
double calculateGrossPay(double hours, double rate);
double calculateOvertimeHours(double hours);
void displayAllEmployees(const vector<int> &ids, const vector<string> &names,
const vector<string> &depts,
                         const vector<double> &hours, const vector<double>
&rates, string header);

void displayDepartments(const vector<int> &ids, const vector<string> &names,
const vector<string> &depts,
                        const vector<double> &hours, const vector<double>
&rates);
double calculateDepartmentPayroll(const vector<string> &depts, const
vector<double> &hours,
                                  const vector<double> &rates, string
targetDept);
int countEmployeesWithOvertime(const vector<double> &hours);
```

```
void displayPayrollSummary(const vector<int> &ids, const vector<string>
&names, const vector<string> &depts,
                           const vector<double> &hours, const vector<double>
&rates);

int main() {

    vector<int> employeeIDs;
    vector<string> surnames;
    vector<string> departments;
    vector<double> hoursWorked;
    vector<double> hourlyRates;

    cout << fixed;

    if (loadEmployeeData(employeeIDs, surnames, departments, hoursWorked,
hourlyRates, DATA_FILE)) {
        cout << "Loaded " << DATA_FILE << endl;
    }else {
        cout << "Failed to load " << DATA_FILE << endl;
        exit(5);
    }

    int userChoice;
    do {
        displayMenu(MENU_ITEMS);
        userChoice = readInt("Enter your choice (1-7): ", 1, 7);

        cout << endl << endl;

        switch (userChoice) {
            case 1:
                // display in original order (sorted by ids)
                sortEmployees(employeeIDs, surnames, departments, hoursWorked,
hourlyRates, SORT_ID_ASCENDING);
                displayAllEmployees(employeeIDs, surnames, departments,
hoursWorked, hourlyRates, "EMPLOYEE PAYROLL REPORT");
                break;
            case 2:
                sortEmployees(employeeIDs, surnames, departments, hoursWorked,
hourlyRates, SORT_NAME_ASCENDING);
                displayAllEmployees(employeeIDs, surnames, departments,
hoursWorked, hourlyRates, "EMPLOYEES SORTED BY NAME (A-Z)");
                break;
            case 3:
                sortEmployees(employeeIDs, surnames, departments, hoursWorked,
hourlyRates, SORT_PAY_DECENDING);
                displayAllEmployees(employeeIDs, surnames, departments,
hoursWorked, hourlyRates, "EMPLOYEES SORTED BY GROSS PAY (Highest to
Lowest)");
                break;
            case 4:
                sortEmployees(employeeIDs, surnames, departments, hoursWorked,
hourlyRates, SORT_DEPARTMENT_ASCENDING);
                displayAllEmployees(employeeIDs, surnames, departments,
hoursWorked, hourlyRates, "EMPLOYEES SORTED BY DEPARTMENT (A-Z)");
```

```
                break;
            case 5:
                sortEmployees(employeeIDs, surnames, departments, hoursWorked,
hourlyRates, SORT_DEPARTMENT_ASCENDING);
                displayDepartments(employeeIDs, surnames, departments,
hoursWorked, hourlyRates);
                break;
            case 6:
                sortEmployees(employeeIDs, surnames, departments, hoursWorked,
hourlyRates, SORT_DEPARTMENT_ASCENDING);
                displayPayrollSummary(employeeIDs, surnames, departments,
hoursWorked, hourlyRates);
                break;
        }

        cout << endl;

    } while (userChoice != 7);

    return 0;
}

/*
 * readInt: Prompts the user for an integer and validates input
 * Parameters
 *   prompt: printed before input is read
 *   min: lowest accepted integer
 *   max: highest accepted integer
 * Return
 *   int: value read
 */
int readInt(string prompt, int min, int max) {
    int input;

    // prime reading
    cout << prompt;
    cin >> input;

    while (input < min || input > max) {
        cout << "Invalid Input";
        cout << prompt;
        cin >> input;
    }

    return input;
}

/*
 * displayMenu: Prints all items in the menu
 */
void displayMenu(const vector<string> &menu_items) {
    cout << endl << "MENU: " << endl;
    for (size_t i = 0; i < menu_items.size(); i++) {
        cout << "  " << (i+1) << ". " << menu_items[i] << endl;
    }
}
```

```
/*
 * loadEmployeeData: read data from file into 5 vectors
 * Parameters
 *   filename: file path to open
 * Returns
 *   bool status, false if the file could not be opened
 */
bool loadEmployeeData(vector<int> &ids, vector<string> &names, vector<string>
&depts,
                      vector<double> &hours, vector<double> &rates, string
filename) {
    ifstream inputFile(filename);

    if (!inputFile.is_open()) {
        return false;
    }

    int numEmployees;
    int employeeID;
    string surname;
    string department;
    double hoursWorked;
    double hourlyRates;

    inputFile >> numEmployees;

    for (int i = 0; i < numEmployees; i++) {
        inputFile >> employeeID >> surname >> department >> hoursWorked >>
hourlyRates;
        ids.push_back(employeeID);

        names.push_back(surname);
        depts.push_back(department);
        hours.push_back(hoursWorked);
        rates.push_back(hourlyRates);
    }

    return true;
}

/*
 * sortEmployees: Selection sort algorithm
 *
 * Parameters
 *   ids
 *   names
 *   depts
 *   hours
 *   rates
 *   sortStyle: integer specifying how it should be sorted, for example
SORT_IDS_ASCENDING
 *   will sort all the arrays so that the ids array is in increasing order
 *
 *   Valid sort styles:
 *     SORT_NAME_ASCENDING
```

```
 *      SORT_DEPARTMENT_ASCENDING
 *      SORT_ID_ASCENDING
 *      SORT_PAY_DECENDING
 */
void sortEmployees(vector<int> &ids, vector<string> &names, vector<string>
&depts,
          vector<double> &hours, vector<double> &rates, int sortStyle) {

    size_t index;
    for (size_t i = 0; i < ids.size(); i++) {
        index = i; // index of element to be swapped

        for (size_t j = i + 1; j < ids.size(); j++) {

            if ((sortStyle == SORT_NAME_ASCENDING && names[j][0] <
names[index][0]) ||
                (sortStyle == SORT_DEPARTMENT_ASCENDING && depts[j][0] <
depts[index][0]) ||
                (sortStyle == SORT_ID_ASCENDING && ids[j] < ids[i]) ||
                (sortStyle == SORT_PAY_DECENDING &&
calculateGrossPay(hours[i], rates[i]) < calculateGrossPay(hours[j],
rates[j]))) {

                index = j;
                }
        }

        // swap elements in all 5 vectors
        swap(ids[i], ids[index]);
        swap(names[i], names[index]);
        swap(depts[i], depts[index]);
        swap(hours[i], hours[index]);
        swap(rates[i], rates[index]);
    }
}

/*
 * calculateGrossPay: calculates the gross pay for an employee, taking into
account overtime bonus
 * Parameters
 *   hours
 *   rate
 * Returns
 *   hours * rate for first 40 hours, hours * rate * OVERTIME_MULTIPLIER for
hours after 40
 */
double calculateGrossPay(double hours, double rate) {
    return hours <= 40 ? hours * rate : (40 * rate) + ((hours-40) * rate *
OVERTIME_MULTIPLIER);
}

/*
 * calculateOvertimeHours: returns number of hours over 40
 * Parameters
 *   hours
 * Returns
```

```
 *   hours over 40, eg if input is 45, returns 5
 */
double calculateOvertimeHours(double hours) {
    return max(hours - 40.0, 0.0);
}

/*
 * displayAllEmployees: displays formatted table of employee data
 * Parameters:
 *   ids
 *   names
 *   depts
 *   hours
 *   rates
 *   header: text to show above the table, useful for describing how the data
is sorted
 */
void displayAllEmployees(const vector<int> &ids, const vector<string> &names,
const vector<string> &depts,
                         const vector<double> &hours, const vector<double>
&rates, string header) {

    cout << header << endl << DOUBLE_BAR << endl;

    cout << setw(5) << "ID"
         << setw(12) << "Name"
         << setw(8) << "Dept"
         << setw(10) << "Reg Hrs"
         << setw(11) << "OT Hours"
         << setw(10) << "Rate"
         << setw(12) << "Gross Pay"
         << endl;

    cout << BAR << endl;

    for (size_t i = 0; i < ids.size(); i++) {
        cout << setprecision(1)
             << setw(5) << ids.at(i)
             << setw(12) << names.at(i)
             << setw(8) << depts.at(i)
             << setw(10) << min(hours.at(i), 40.0)
             << setw(11) << calculateOvertimeHours(hours.at(i));


        cout << setprecision(2)
             << setw(5) << "$" << rates.at(i)
             << setw(5) << "$" << calculateGrossPay(hours.at(i), rates.at(i))
             << endl;
    }
}

/*
 * displayDepartments: similar to displayAllEmployees, but groups employees
by department and shows
 * pay for the entire department
 * Parameters
```

```
 *    ids
 *    names
 *    depts
 *    hours
 *    rates
 */
void displayDepartments(const vector<int> &ids, const vector<string> &names,
const vector<string> &depts,
                        const vector<double> &hours, const vector<double>
&rates) {

    cout << "PAYROLL BY DEPARTMENT" << endl << DOUBLE_BAR << endl;

    cout << fixed;

    cout << setw(5) << "ID"
         << setw(12) << "Name"
         << setw(18) << "Reg Hrs"
         << setw(11) << "OT Hours"
         << setw(10) << "Rate"
         << setw(12) << "Gross Pay"
         << endl;

    cout << BAR << endl;

    string department;
    double deptTotal = 0;
    double companyTotal = 0;

    for (size_t i = 0; i < ids.size(); i++) {

        if (department != depts.at(i)) {
            department = depts.at(i);
            cout << "Department: " << department << endl << BAR << endl;
        }

        cout << setprecision(1)
             << setw(5) << ids.at(i)
             << setw(12) << names.at(i)
             << setw(18) << min(hours.at(i), 40.0)
             << setw(11) << calculateOvertimeHours(hours.at(i));


        cout << setprecision(2)
             << setw(5) << "$" << rates.at(i)
             << setw(5) << "$" << calculateGrossPay(hours.at(i), rates.at(i))
             << endl;

        if (i == ids.size()-1 || department != depts.at(i+1)) {
            deptTotal = calculateDepartmentPayroll(depts, hours, rates,
department);
            companyTotal += deptTotal;
            cout << setprecision(2) << setw(61) << "Department total: $" <<
deptTotal << endl;
        }
    }
```

```
    cout << DOUBLE_BAR << endl << "Company Total: $" << companyTotal << endl;
}

/*
 * calculateDepartmentPayroll: find total pay for all employees in a
department
 * Parameters
 *   depts
 *   hours
 *   rates
 *   targetDept: department to sum
 * Returns
 *   gross pay for the department
 */
double calculateDepartmentPayroll(const vector<string> &depts, const
vector<double> &hours,
                                  const vector<double> &rates, string
targetDept) {
    double totalDepartment = 0;

    for (size_t i = 0; i < depts.size(); i++) {
        if (depts.at(i) == targetDept) {
            totalDepartment += calculateGrossPay(hours.at(i), rates.at(i));
        }
    }

    return totalDepartment;
}

/*
 * countEmployeesWithOvertime
 * Returns
 *   number of employees who worked more than 40 hours
 */
int countEmployeesWithOvertime(const vector<double> &hours) {
    int count = 0;
    for (double hour : hours) {
        if (hour > 40.0) {
            count ++;
        }
    }
    return count;
}

/*
 * displayPayrollSummary: displays a detailed breakdown of employee pay data.
 * Parameters
 *   ids
 *   names
 *   depts
 *   hours
 *   rates
 */
void displayPayrollSummary(const vector<int> &ids, const vector<string>
&names, const vector<string> &depts,
```

```
                            const vector<double> &hours, const vector<double>
&rates) {

    int employees = ids.size();
    int withOT = countEmployeesWithOvertime(hours);
    int withoutOT = employees - withOT;

    double totalHoursNoOT = 0, totalHoursOT = 0, regularPay = 0, overtimePay =
0;
    double highest = 0, lowest = 0;
    int highestIndex, lowestIndex = 0;
    for (size_t i = 0; i < hours.size(); i++) {
        double hour = hours.at(i);
        double totalPay = calculateGrossPay(hour, rates.at(i));

        // find the highest and lowest paid employees
        if (totalPay > highest) {
            highest = totalPay;
            highestIndex = i;
        }
        if (totalPay < lowest || i == 0) {
            lowest = totalPay;
            lowestIndex = i;
        }

        // sum of regular pay
        totalHoursNoOT += min(hour, 40.0);
        regularPay += min(hour, 40.0) * rates.at(i);

        // sum of overtime pay
        if (hours[i] > 40.0) {
            totalHoursOT += hour - 40.0;
            overtimePay += (hour - 40.0) * rates.at(i) * OVERTIME_MULTIPLIER;
        }
    }
    double totalHours = totalHoursNoOT + totalHoursOT;
    double grossPay = regularPay + overtimePay;


    cout << "PAYROLL SUMMARY REPORT" << endl << DOUBLE_BAR << endl;

    cout << endl << "  EMPLOYEE STATISTICS" << endl << BAR_SMALL << endl <<
setprecision(2)
         << "  Total Employees: " << employees
         << "  Employees with Overtime: " << withOT << endl
         << "  Employees without OT: " << withoutOT << endl;


    cout << endl << "  HOURS SUMMARY" << endl << BAR_SMALL << endl <<
setprecision(1)
         << "  Total Regular Hours: " << totalHoursNoOT << endl
         << "  Total Overtime Hours: " << totalHoursOT << endl
         << "  Total Hours Worked: " << totalHours << endl
         << "  Average Hours/Employee: " << (totalHours / employees) << endl;
```

```
    cout << endl << "  PAY SUMMARY" << endl << BAR_SMALL << endl <<
setprecision(2)
         << "  Total Regular Pay: $" << regularPay << endl
         << "  Total Overtime Pay: $" << overtimePay << endl
         << "  Total Gross Payroll: $" << grossPay << endl
         << "  Highest Paid: " << names.at(highestIndex) << " $" << highest <<
endl
         << "  Lowest Paid: " << names.at(lowestIndex) << " $" << lowest <<
endl
         << "  Average Pay: $" << ((regularPay + overtimePay) / employees) <<
endl;

    cout << endl << "  DEPARTMENT BREAKDOWN" << endl << BAR_SMALL << endl;
    int employeesInDept = 0;
    for (size_t i = 0; i < depts.size(); i++) {
        employeesInDept++;
        if (i == depts.size()-1 || depts.at(i) != depts.at(i+1)) {
            double deptPay = calculateDepartmentPayroll(depts, hours, rates,
depts.at(i));

            cout << "  " << depts.at(i) << ": " << employeesInDept << "
employees $"
                           << deptPay << " (" << (100*deptPay/grossPay) << "%)"
<< endl;

            employeesInDept = 0;
        }
    }

}
```